

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I M. Ajitha (192525391) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student:M.Ajitha

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

When a machine learning model is trained using gradient descent, the learning rate controls how much the model updates its parameters after each iteration. Choosing a high learning rate at the beginning and then gradually decreasing it (learning rate decay) is a common optimization strategy.

Effect of a High Initial Learning Rate

A high learning rate allows the model to take large steps during the early stages of training. This helps the algorithm move quickly toward the region where the minimum loss exists instead of making very small updates. As a result, the model usually reaches a good solution faster than using a small learning rate from the start.

However, if the learning rate is too high, the updates may become unstable. Instead of moving smoothly toward the minimum, the algorithm may overshoot the optimal point, oscillate around it, or even fail to converge.

Effect of Gradually Decreasing the Learning Rate

As training progresses, reducing the learning rate makes the parameter updates smaller and more precise. This allows the model to fine-tune its weights and settle closer to the minimum of the convex loss function. Learning rate decay improves stability and helps achieve better accuracy near convergence.

When This Strategy Outperforms a Constant Learning Rate

A high initial learning rate followed by gradual decay can perform better than a constant learning rate under the following conditions:

- When the loss function is convex, allowing gradient descent to move safely toward the global minimum.
- When the initial learning rate is large enough to speed up learning but not so large that training becomes unstable.
- When the learning rate is reduced gradually, giving the model enough time to make accurate adjustments near the minimum.
- When faster convergence and efficient training are required, especially for large datasets.

Compared to a constant learning rate:

- A small constant learning rate is stable but often converges slowly.
- A large constant learning rate may never converge because it keeps taking large steps.
- A decaying learning rate combines the advantages of both: fast progress at the beginning and stable convergence at the end.

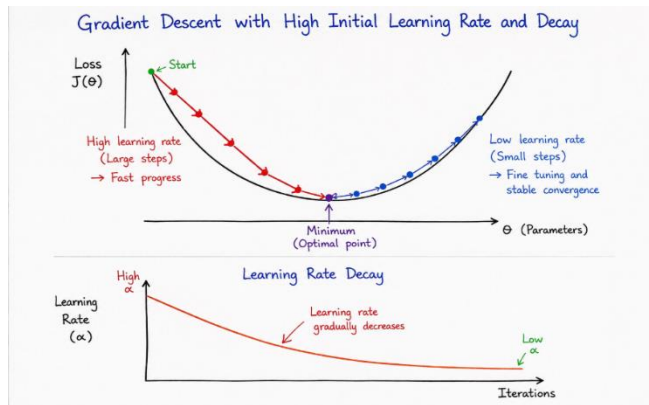
Possible Risks

Although this strategy is effective, it also has some risks:

- If the initial learning rate is too high, the model may oscillate or diverge instead of converging.

- If the learning rate decreases too quickly, the model may stop learning before reaching the optimal solution.
- If the learning rate decreases too slowly, convergence may remain slow and require more iterations.
- Selecting an appropriate decay schedule requires careful tuning, as poor settings can reduce model performance.

Gradient Descent Training



2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Assume that the dataset contains n observations

$$x_1, x_2, \dots, x_n$$

from a Gaussian distribution with unknown mean μ and known variance σ^2 .

Step 1: Likelihood Function

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Step 2: Log-Likelihood

$$\log L(\mu) = -n \log(\sqrt{2\pi\sigma^2}) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3: Differentiate and Set to Zero

$$\frac{d}{d\mu} \log L(\mu) = 0$$

$$\sum_{i=1}^n (x_i - \mu) = 0$$

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

Hence, the Maximum Likelihood Estimator (MLE) of the mean is the sample mean.

Effect of Outliers

- The sample mean uses every observation equally.
- A single extreme value (outlier) can shift the estimated mean significantly.
- Therefore, the MLE is sensitive to outliers.

Effect of Increasing Sample Size

- As the number of samples increases, the estimate becomes more stable.
- By the Law of Large Numbers,
 $\hat{\mu} \rightarrow \mu \text{ as } n \rightarrow \infty$

This means the estimated mean gets closer to the true mean.

Robustness of MLE

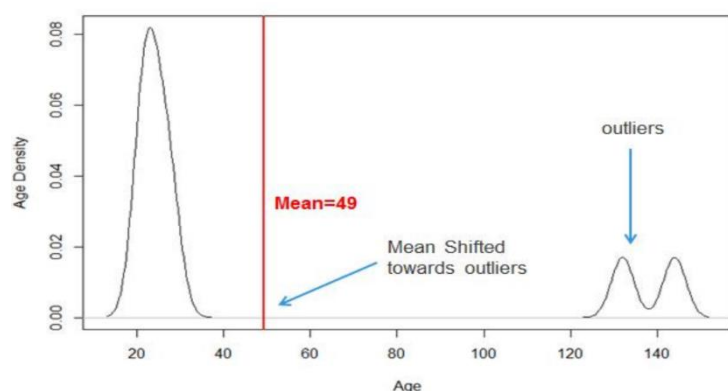
- **Advantages:** Simple, efficient, and accurate for normally distributed data.
- **Limitation:** Not robust because outliers can greatly affect the estimate.

Conclusion

The MLE of the mean is

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

It becomes more accurate as the sample size increases, but it is not robust to outliers, since extreme values can noticeably change the estimated mean.



3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer

The dataset contains 10,000 features but only 500 training samples. This is known as a high-dimensional dataset, where the number of features is much larger than the number of samples. Training a model directly on such data can lead to poor performance and overfitting.

Challenges

- **Overfitting:** The model may memorize the training data instead of learning useful patterns.
- **High Computational Cost:** Processing 10,000 features requires more memory and computation.
- **Irrelevant Features:** Many features may not contribute to prediction and can reduce model accuracy.
- **Curse of Dimensionality:** As the number of features increases, learning meaningful relationships becomes difficult.

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.

- Normalize or standardize the features.

Standardization formula:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x = feature value
- μ = mean
- σ = standard deviation

Step 2: Feature Selection

Remove irrelevant or redundant features using methods such as SelectKBest or L1 (Lasso) feature selection.

This keeps only the most important features, reducing model complexity

Step 3: Dimensionality Reduction

Apply Principal Component Analysis (PCA) to reduce the number of features while preserving most of the important information.

PCA transforms the original features into a smaller number of principal components.

Step 4: Train the Classifier

Train a classifier such as Logistic Regression or Support Vector Machine (SVM) with L2 Regularization.

Regularized loss function:

$$J(\theta) = L(\theta) + \lambda \sum_{j=1}^n \theta_j^2$$

where:

- $L(\theta)$ = training loss
- λ = regularization parameter

Regularization penalizes large weights and reduces overfitting.

Step 5: Model Evaluation

Evaluate the model using k-fold cross-validation to check its performance on unseen data.

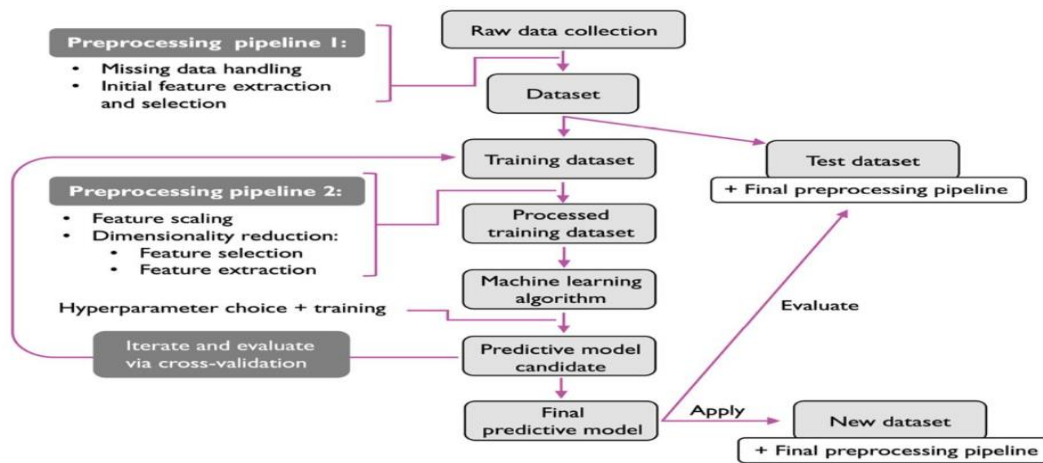
Accuracy formula:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

How This Approach Reduces Overfitting

- Feature Selection removes unnecessary features.
- PCA reduces the dimensionality while retaining useful information.
- Regularization prevents the model from learning noise.
- Cross-validation ensures the model generalizes well to new data.

Together, these steps produce a simpler and more reliable model.



4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer

A Multilayer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. In this network, the hidden layer uses the sigmoid activation function to learn complex relationships between inputs and outputs.

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns a different feature from the input data. By increasing the number of hidden neurons, the network can capture more complex patterns and decision boundaries.

The sigmoid activation function is

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The output of a hidden neuron is

$$h = \sigma(Wx + b)$$

where:

- W = weight
- x = input
- b = bias

The final output of the MLP is

$$y = \sigma\left(\sum_{i=1}^n w_i h_i + b\right)$$

As the number of hidden neurons increases, the network can combine more learned features, making it capable of representing more complicated functions.

Why More Hidden Neurons Do Not Always Improve Performance

Although adding more hidden neurons increases the learning capacity, it does not always produce better results.

Overfitting

- A large network may memorize the training data instead of learning general patterns.
- It performs very well on training data but poorly on new or unseen data.

Generalization

- A good model should perform well on both training and testing data.
- Too many hidden neurons reduce the model's ability to generalize because it starts fitting noise instead of useful information.

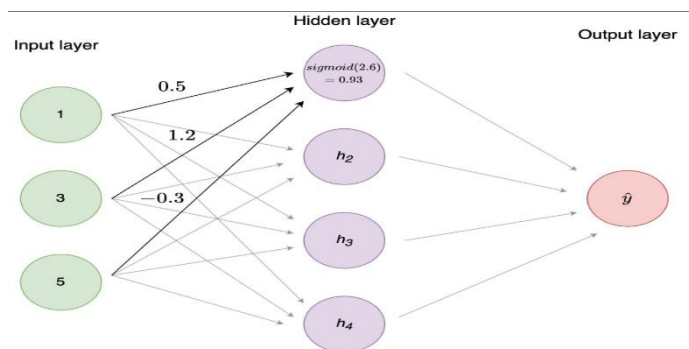
Other Issues

- Training time and computational cost increase.
- More parameters require more memory.
- Choosing the correct number of hidden neurons is important for achieving good performance

How to Avoid Overfitting

- Use an appropriate number of hidden neurons.
- Apply regularization or dropout.
- Use early stopping during training.
- Train with sufficient data and validate the model using a validation set.

These techniques improve the model's ability to generalize.



5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the curse of dimensionality impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically

Introduction

The curse of dimensionality occurs when the number of input features becomes very large. In high-dimensional data, samples become sparse, making it difficult for a deep neural network to learn meaningful patterns. This affects backpropagation, SGD convergence, and model generalization

Impact on Gradient Updates in Backpropagation

During backpropagation, gradients are calculated to update the network weights.

In high-dimensional data:

- Many features are irrelevant or noisy.

- Gradient values become noisy and unstable.
- Weight updates become less accurate.

Formula 1: Gradient Update

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Where:

- θ = model parameters (weights)
- η = learning rate
- $\nabla J(\theta)$ = gradient of the loss function

Analysis:

When the feature dimension is very high, the gradient estimate becomes noisy. Therefore, weight updates are less reliable, making training unstable.

2. Impact on Convergence Speed of SGD

SGD updates weights using only one or a few training samples at a time.

In high-dimensional space:

- Data points become sparse.
- Gradient directions vary greatly.
- SGD requires more iterations to reach the minimum.

Result:

- Slower convergence
- Longer training time
- More oscillations during optimization

3. Impact on Model Generalization

Generalization means performing well on unseen data.

In high-dimensional datasets:

- The model may memorize training data.
- It learns noise instead of useful patterns.
- This causes overfitting.

Formula 2: Regularized Loss Function

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda \|\theta\|_2^2$$

Where:

- $J(\theta)$ = original loss
- λ = regularization parameter
- $\|\theta\|_2^2$ = L2 penalty

Analysis:

Regularization reduces overly large weights and helps the model generalize better.

Techniques to Overcome the Curse of Dimensionality

Technique 1: Dimensionality Reduction (PCA / Autoencoders)

Justification:

- Removes irrelevant and redundant features.
- Reduces feature space from high dimension to lower dimension.
- Produces more stable gradients.
- Speeds up SGD convergence.
- Improves model accuracy on new data.

Formula 3

$$d \ll D$$

Where:

- D = original number of features
- d = reduced number of features

Technique 2: Regularization (L2, Dropout, Batch Normalization)

Justification:

- Prevents overfitting.
- Reduces model complexity.
- Improves generalization.
- Makes training more stable.

